Дружественность не наследуется

```
class A {
    int y = 0;
protected:
    int x = 1;
public:
    int z = 2;

    friend int main();
};

class B: A {
    int xb = 3;
public:
    int yb = 4;
    void f() {
        x = 2;  //Разрешено
        y = 3;  //Запрещено, это закрытое поле
    }
};

int main () {
    B b;
    std::cout << b.yb; //Разрешено
    std::cout << b.y;  //Запрещено
    A a;
    std::cout << a.y << ' ' << a.x;   //Разрешено, потому что main – друг
};
```

```
class A {
    int y = 0;
protected:
    int x = 1;
public:
    int z = 2;

    friend class B;
};


class B: A {
    int xb = 3;
public:
    int yb = 4;
    void f() {
        x = 2;  //Разрешено
        y = 3;  //Разрешено, потому что друг
    }
};


int main () {
    B b;
    std::cout << b.yb;
    std::cout << b.y;   //Запрещено, закрытое наследование
    A a;
    std::cout << a.y << ' ' << a.x; //Запрещено, private y и protected x
};
```

Области видимости

```
class A {
    int y = 0;
    int x = 1;
public:
    int z = 2;
    void f (int) {
        std::cout << "A\n";
    }
};

class B: public A {
    int x = 3;
public:
    int y = 4;
    int z = 5;
    void f () {
        std::cout << "B\n";
    }
};

int main () {
    B b;
    A a;
    std::cout << b.z << ' ' << a.z << '\n';
    b.f(0); //попытка вызова функции из B::
};
```

Определение области видимости

```
void f (double) {
    std::cout << "out\n";
}
class A {
    int y = 0;
public:
    int x = 1;
    int z = 2;
    void f (double) {
        std::cout << "A\n";
    }
};
class B: public A {
    int xb = 3;
public:
    int y = 4;
    int z = 5;
    void f (double) {
        ::f(2);
        std::cout << "B\n";
    }
};
int main () {
    B b;
    b.f(3);
    b.A::f(0);
    f(2); //аналогично ::f(2);
};
```

Доступ к базовому классу

```
class A {
    int y = 0;
protected:
    int x = 1;
public:
    int z = 2;
};


class B: A {
    int xb = 3;
public:
    int yb = 4;
    void f(A& a) {
        x = 2;
    }
};
class C: public B {
    void f1(::A& a) { //иначе будет поиск типа A в контексте C, а этот тип из C недоступен
        a.z = 3;
    }
};
```

using

```
class A {
    int y = 0;
public:
    int x = 1;
    int z = 2;
    void f (int) {
        std::cout << "A\n";
    }
};


class B: public A {
    int xb = 3;
public:
    int y = 4;
    int z = 5;
    using A::f;
    void f (double) {
        std::cout << "B\n";
    }
};


int main () {
    B b;
    b.f(0);   //A
};
```

using (продолжение)

```
class A {
    int y = 0;
protected:
    void f (int) {
        std::cout << "A\n";
    }
public:
    int x = 1;
    int z = 2;

};
class B: public A {
    int xb = 3;
public:
    int y = 4;
    int z = 5;
    using A::f;
    void f (double) {
        std::cout << "B\n";
    }
};

int main () {
    B b;
    b.f(0);   //A
};
```

using для управления доступом

```
class A {
    int y = 0;
public:
    int x = 1;
    int z = 2;
    void f (int) {
        std::cout << "A\n";
    }
};

class B: public A {
    int xb = 3;
    using A::f;
public:
    int y = 4;
    int z = 5;
    void f (double) {
        std::cout << "B\n";
    }
};

int main () {
    B b;
    b.f(0);                                   'f' is a private member of 'B'
    b.A::f(2);
};
```

using переменных

```
class A {
    int y = 0;
public:
    int x = 1;
    int z = 2;
    void f (double) {
        std::cout << "A\n";
    }
};

class B: public A {
    int x = 3;
    using A::f;
public:
    using A::x;
    int y = 4;
    int z = 5;
    void f (double) {
        std::cout << "B\n";
    }
};

int main () {
    B b;
    b.f(0);
    b.A::f(2);
};
```

Target of using declaration conflicts with declaration already in scope

Порядок подбора подходящей функции:

1. Выбирается область видимости
2. Выполняется выбор подходящей функции, перегрузка
3. Проверка доступа

Правила для специальных методов

- Если в конструкторе производного класса явный вызов конструктора базового класса отсутствует, автоматически **вызывается конструктор базового класса по умолчанию**. То есть если такого конструктора в базовом классе нет, будет ошибка.
- Для иерархии, состоящей из нескольких классов, конструкторы базового класса вызываются, начиная с **самого верхнего уровня**. После этого выполняются конструкторы элементов класса, являющихся объектами, а затем - конструктор класса.
- В случае нескольких базовых классов, их конструкторы вызываются **в порядке объявления**.
- **Для деструкторов** эти правила справедливы, но порядок вызова обратный - **сначала производный класс, затем базовый класс**. Не требуется явно вызывать деструкторы, поскольку они будут вызваны автоматически.

Вызов конструктора базового класса

```
class Employee {
    char* name = nullptr;
    char* surname = nullptr;
    char date[11] {"01.01.1901"};
public:
    Employee (const char* n, const char *s, const char d[]): name(new char[strlen(n)+1]), surname(new char[strlen(s)+1]) {
        memcpy(name, n, strlen(n));
        memcpy(surname, s, strlen(s));
        memcpy(date, d, strlen(d));
    }
};

class Programmer: public Employee {
    char* team = nullptr;
public:
    Programmer(const char* n, const char *s, const char d[], const char* t):
        team(new char[strlen(t)+1]){
            memcpy(team, t, strlen(t));
    }
};
int main () {
    Programmer p ("test", "test", "01.02.2025", "test");
};
```

Constructor for 'Programmer' must explicitly initialize the base class 'Employee' which does not have a default constructor

```
class Employee {
    char* name = nullptr;
    char* surname = nullptr;
    char date[11] {"01.01.1901"};
public:
    Employee () {std::cout << "Employee\n";}
    Employee (const char* n, const char *s, const char d[]):name(new char[strlen(n)+1]), surname(new char[strlen(s)+1]){
            memcpy(name, n, strlen(n));
            memcpy(surname, s, strlen(s));
            memcpy(date, d, strlen(d));
            std::cout << "Employee " << name << '\n';
    }
    ~Employee () {
        delete [] name;
        delete [] surname;
        std::cout << "~Employee\n";
    }
};
class Programmer: public Employee {
    char* team = nullptr;
public:
    Programmer () {std::cout << "Programmer\n";}
    Programmer(const char* n, const char *s, const char d[], const char* t): team(new char[strlen(t)+1]){
            memcpy(team, t, strlen(t));
            std::cout << "Programmer " << team << '\n';
    }
    ~Programmer () {
        delete [] team;
        std::cout << "~Programmer\n";
    }
};
int main () {
    Programmer Vasya;
    Programmer Test("test", "test", "01.02.2025", "test");}
```

Что будет выведено на предыдущем слайде?

- Employee
- Programmer
- Employee
- Programmer test

“наследование” конструктора

```
class Employee {
    char* name = nullptr;
    char* surname = nullptr;
    char date[11] {"01.01.1901"};
public:
    Employee () {std::cout << "Employee\n";}
    Employee (const char* n, const char *s, const char d[]):
          name(new char[strlen(n)+1])
        , surname(new char[strlen(s)+1]) {
            memcpy(name, n, strlen(n));
            memcpy(surname, s, strlen(s));
            memcpy(date, d, strlen(d));
            std::cout << "Employee " << name << '\n';
    }
    ~Employee () {
        delete [] name;
        delete [] surname;
        std::cout << "~Employee\n";
    }
};
class Programmer: public Employee {
    int grade = 13;
public:
    using Employee::Employee;
};
int main () {
    Programmer Vasya;
    Programmer Test("test", "test", "01.02.2025");
};
```

Конструктор копии? (здесь вызовется конструктор копии базового, а у производного копирующий конструктор будет создан по умолчанию, то есть произведено побайтовое копирование)

```
class Employee {
    char* name = nullptr;
    char* surname = nullptr;
    char date[11] {"01.01.1901"};
public:
    Employee () {std::cout << "Employee\n";}
    Employee (const char* n, const char *s, const char d[]): name(new char[strlen(n)+1]), surname(new char[strlen(s)+1]) {
            memcpy(name, n, strlen(n));
            memcpy(surname, s, strlen(s));
            memcpy(date, d, strlen(d));
            std::cout << "Employee " << name << '\n';
    }
    Employee (const Employee& other): Employee(other.name, other.surname, other.date) {}
    ~Employee () {
        delete [] name;
        delete [] surname;
        std::cout << "~Employee\n";
    }
};
class Programmer: public Employee {
    char* team = nullptr;
public:
    using Employee::Employee;
    Programmer(const char* n, const char *s, const char d[], const char* t): Employee(n, s, d), team(new char[strlen(t)+1]) {
                memcpy(team, t, strlen(t));
                std::cout << "Programmer " << team << '\n';
        }
    ~Programmer(){
        delete [] team;                                                    Thread 1: signal SIGABRT
    }
};
int main () {
    Programmer Test("test", "test", "01.02.2025", "team");
    Programmer Vasya = Test;
};
```

```
class Employee {
    char* name = nullptr;
    char* surname = nullptr;
    char date[11] {"01.01.1901"};
public:
    Employee () {}
    Employee (const char* n, const char *s, const char d[]): name(new char[strlen(n)+1]), surname(new char[strlen(s)+1]) {
            memcpy(name, n, strlen(n));
            memcpy(surname, s, strlen(s));
            memcpy(date, d, strlen(d));
    }
    Employee (const Employee& other): Employee(other.name, other.surname, other.date) {}
    ~Employee () {
        delete [] name;
        delete [] surname;
    }
};
class Programmer: public Employee {
    char* team = nullptr;
public:
    using Employee::Employee;
    Programmer(const char* n, const char *s, const char d[], const char* t): Employee(n, s, d), team(new char[strlen(t)+1]) {
                memcpy(team, t, strlen(t));
        }
    Programmer (const Programmer& other): team(new char[strlen(other.team)+1]) {
        memcpy(team, other.team, strlen(other.team));
    }
    ~Programmer(){
        delete [] team;
    }
};
int main () {
    Programmer Test("test", "test", "01.02.2025", "team");
    Programmer Vasya = Test;
};
```

А здесь при копировании будет вызван базовый конструктор по умолчанию, то есть скопируется только производная часть, а базовая будет «по умолчанию»

Здесь будет «полное» копирование объекта

```
class Employee {
    char* name = nullptr;
    char* surname = nullptr;
    char date[11] {"01.01.1901"};
public:
    Employee () {}
    Employee (const char* n, const char *s, const char d[]): name(new char[strlen(n)+1]), surname(new char[strlen(s)+1]) {
            memcpy(name, n, strlen(n));
            memcpy(surname, s, strlen(s));
            memcpy(date, d, strlen(d));
    }
    Employee (const Employee& other): Employee(other.name, other.surname, other.date) {}
    ~Employee () {
        delete [] name;
        delete [] surname;
    }
};
class Programmer: public Employee {
    char* team = nullptr;
public:
    using Employee::Employee;
    Programmer(const char* n, const char *s, const char d[], const char* t): Employee(n, s, d), team(new char[strlen(t)+1]) {
                memcpy(team, t, strlen(t));
        }
    Programmer (const Programmer& other): Employee (other), team(new char[strlen(other.team)+1]) {
        memcpy(team, other.team, strlen(other.team));
    }
    ~Programmer(){
        delete [] team;
    }
};
int main () {
    Programmer Test("test", "test", "01.02.2025", "team");
    Programmer Vasya = Test;
};
```

Правило трёх при наследовании

Если в производном классе есть нетривиальные поля, то для него обязательно необходимо определять конструктор копирования, оператор присваивания и деструктор. Причём, рекомендуется предусмотреть вызов соответствующих методов в родительском классе (кроме деструктора).

Если в производном классе нетривиальные поля отсутствуют, копирование, присваивание и удаление объекта будут выполнены автоматически, при этом, при необходимости, будут автоматически вызваны соответствующие методы родительского класса.

Приведение типов

```
class Box {  // тип "коробка"
    int width = 5;
    int height = 8;
public:
    Box() {}
    Box(const Box& b): width (b.width), height (b.height){}
    void SetWidth(int w) { width = w; }
    void SetHeight(int h) { height = h; }
};
class ColoredBox : public Box { // "цветная коробка"
    int color = 8;
public:
    ColoredBox() {}
    ColoredBox (const ColoredBox& other):Box(other), color(other.color){}
};


void f(Box& b) { // Разрешено, будет «срез»
    b.SetWidth(3);
    b.SetHeight(5);
}
int main () {
    ColoredBox cbox;
    f(cbox);
    Box b = cbox; //Разрешено
    ColoredBox cbox1 = b; //Запрещено
};
```

Приведение типов. Производный к базовому.

```
class A {};
class B : public A {};
```

	По значению	По указателю	По ссылке
Неявное приведение	+	+	+
Явное приведение	+	+	+

```
void f (A& a) {}
int main() {
    A a;
    A* pa = &a;
    B b;
    B* pb = &b;
    a = b;
    pa = pb;
    pa = &b;
    f(b);
    pa = static_cast<A*>(pb);
    pa = static_cast<A*>(&b);
    f(static_cast<A&>(b));
    pa = (A*)(pb);  // так лучше не делать
}
```

Приведение типов. Производный к базовому.

```
class A {};
class B : A {};
```

	По значению	По указателю	По ссылке
Неявное приведение	-	-	-
Явное приведение	-	-	-

```
void f (A& a) {}
int main() {
    A a;
    A* pa = &a;
    B b;
    B* pb = &b;
    a = b;    // запрещено
    pa = pb; // запрещено
    pa = &b; // запрещено
    f(b);     // запрещено
    pa = static_cast<A*>(pb);  // запрещено
    pa = static_cast<A*>(&b);  // запрещено
    f(static_cast<A&>(b));     // запрещено
    pa = (A*)(pb);  // так лучше не делать
}
```

Приведение типов. Базовый к производному

```
class A {};
class B : public  A {};
```

	По значению	По указателю	По ссылке
Неявное приведение	-	-	-
Явное приведение	-	+ (UB)	+ (UB)

```
void f (B& a) {}
int main() {
    A a;
    A* pa = &a;
    B b;
    B* pb = &b;
    b = a;       // запрещено
    pb = pa;     // запрещено
    pb = &a;     // запрещено
    f(a);        // запрещено
    pb = static_cast<B*>(pa); // допустимо
    pb = static_cast<B*>(&a); // допустимо
    f(static_cast<B&>(a));    // допустимо
    pb = (B*)(pa); // так лучше не делать
}
```

Приведение типов. Базовый к производному

```
class A {};
class B : A {};
```

	По значению	По указателю	По ссылке
Неявное приведение	-	-	-
Явное приведение	-	-	-

```
void f (B& a) {}
int main() {
    A a;
    A* pa = &a;
    B b;
    B* pb = &b;
    b = a;       // запрещено
    pb = pa;     // запрещено
    pb = &a;     // запрещено
    f(a);        // запрещено
    pb = static_cast<B*>(pa); // запрещено
    pb = static_cast<B*>(&a); // запрещено
    f(static_cast<B&>(a));    // запрещено
    pb = (B*)(pa); // так лучше не делать
}
```

Полиморфизм

Опять про приведение типов

```
class  A {
public:
    int a = 3;
    void f(){std::cout << "A\n";}
};


class B: public A{
public:
    int b = 5;
    void f(){std::cout << "B\n";}
};


int main() {
    B b;
    A& a = b;
    a.f();  //?
}
```

При статическом связывании вызов функции через указатель **определяется исключительно типом указателя**, а не объектом, на который он указывает. При статическом связывании **вызовы функций фиксируются до выполнения программы на этапе компиляции**.

```
class A {
 public:
  A() = default;
  void print() const { std::cout << "print() from A" << std::endl; }
};


class B: public A {
 public:
  B() : A() {}
  void print() const { std::cout << "print() from B" << std::endl; }
};


int main() {
  A* ent = new B();
  ent->print();  // print() from A
  delete ent;
}
```

Виртуальные функции. Они могут быть переопределены в производных классах

```
class A {
 public:
  A() = default;
  virtual void print() const { std::cout << "print() from A" << std::endl; }
};


class B: public A {
 public:
  B() : A() {}
  void print() const { std::cout << "print() from B" << std::endl; }
};


int main() {
  A* ent = new B();
  ent->print();  // print() from B
  ent->A::print(); // print() from A
  delete ent;
}
```

Полиморфный класс

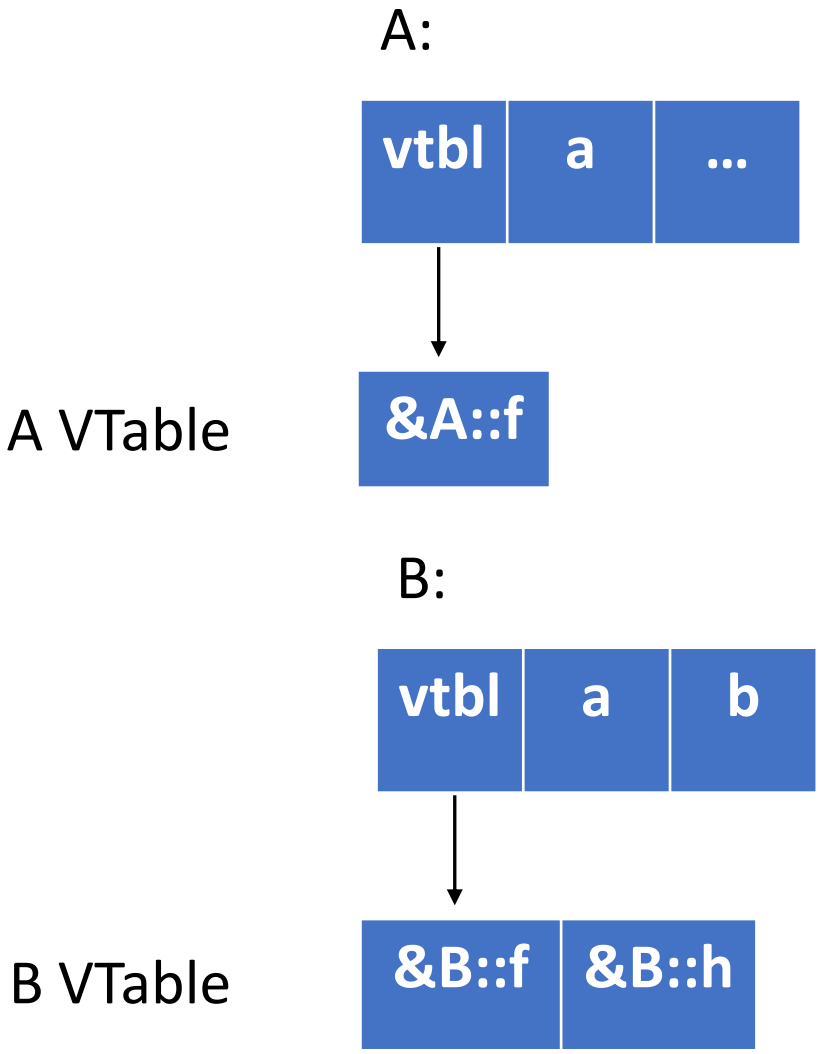
При **динамическом связывании то, какую функцию какого типа вызвать, решается на этапе выполнения**.
Класс, который определяет или наследует виртуальную функцию, называется **полиморфным**. То есть в данном случае A и B являются полиморфными классами.

- Чтобы функция попадала под **динамическое связывание**, в производном классе она должна иметь тот же самый набор параметров и возвращаемый тип, что и в базовом классе.
- Если же функция имеет разный набор параметров или несоответствие по константности, то мы будем иметь дело со скрытием функций, а не переопределением. И тогда будет применяться **статическое связывание**.

Расположение в памяти (в следующей лекции рассмотрим подробнее)

```
class A {
    int a = 2;
public:
    A() = default;
    virtual void f() const {}
    void g() {}
};


class B: public A {
    int b = 3;
public:
    B() : A() {}
    void f() const override {}
    virtual void h() {}
};
```



vtable

Механизм виртуальности используется, когда виртуальная функция вызывается через указатель или ссылку на базовый класс.

- Любой класс, который использует виртуальные функции (или дочерний класс, родительский класс которого использует виртуальные функции), имеет свою собственную **виртуальную таблицу**. Это статическая структура, которая создается компилятором во время компиляции.
- Виртуальная таблица содержит **по одной записи на каждую виртуальную функцию**, которая может быть вызвана объектами класса. Каждая запись в этой таблице — это **указатель на функцию, указывающий на метод, доступный объекту этого класса**.
- Когда мы связываем объект производного класса с указателем базового класса, переменная vtbl указывает на vtable производного класса. Это **присвоение гарантирует, что будет вызвана нужная виртуальная функция**.

https://shaharmike.com/cpp/

override

Ключевое слово override служит двум целям:

1. Показывает, что "это виртуальный метод, который **переопределяет** виртуальный метод базового класса".
2. Компилятор также знает, что это переопределение, поэтому он может **"проверить"**, что вы не изменяете/не добавляете новые методы, которые, по вашему мнению, являются переопределениями.

```
class A {
public:
    A() = default;
    virtual void print() const { std::cout << "print() from A" << std::endl; }
};
class B: public A {
public:
    B() : A() {}
    void print() const override { std::cout << "print() from B" << std::endl; }
};

class C: public B {
public:
    void print() override { std::cout << "print() from C" << std::endl; }  2 ⊗ Non-virtual member function marked 'override' hides virtual memb...
};
int main() {
    C c;
    A& a = c;
    a.print();  // print() from C
}
```

override

override явно указывает компилятору на то, что мы хотим переопределить функцию. Если переопределяемая функция в производном классе не соответствует виртуальной функции в базовом классе по списку параметров, возвращаемому типу, константности, или в базовом классе вообще нет функции с таким именем, то **компилятор при компиляции сгенерирует ошибку**.

Если же override не указать, то компилятор будет считать, что речь идет о скрытии функции, и никаких ошибок не будет генерировать.

final

При использовании в объявлении или определении виртуальной функции final спецификатор гарантирует, что функция является виртуальной, и указывает, что она не может быть переопределена производными классами. В противном случае программа неправильно сформирована (генерируется ошибка времени компиляции).

P.S. Отмечайте виртуальные функции базовые - как virtual, производные - как override.

PPS. Класс тоже можно пометить final, что запрещает дальнейшее его наследование

final

```
class A {
public:
    A() = default;
    virtual void print() const { std::cout << "print() from A" << std::endl; }
};
class B: public A {
public:
    B() : A() {}
    void print() const final { std::cout << "print() from B" << std::endl; }
};
class C: public B {
public:
    void print() const { std::cout << "print() from C" << std::endl; }
};
int main() {
    C c;
    A& a = c;
    a.print();  // print() from C
}
```

Declaration of 'print' overrides a 'final' function